

Math and Audio Visualization App

Research Brief + Claude Build Prompt

Prepared for planning and product direction

Date: April 24, 2026

1. Executive Summary

This project sits between a graphing calculator, a simulation sandbox, and an audio-reactive visual instrument. The core opportunity is not to build a tool that “does all of math,” but to build a tool that makes equations, waveforms, and mathematical systems feel alive in real time. The strongest version 1 is an app where a user enters a function or waveform, sees the conventional graph immediately, and also sees a real-time visual interpretation driven by the same signal or equation.

Thesis

A compelling first product is a real-time equation and waveform visualizer that turns mathematical structure into motion. That is more focused, more buildable, and more differentiated than a generic “math simulation platform.”

Why this is promising

- Graphing tools already exist, but most are not cinematic, audio-reactive, or performance-oriented.
- Audio visualizers already exist, but most are not equation-first or designed for mathematical exploration.
- Simulation demos already exist, but most are either educational-only or code-first rather than interactive creative tools.
- The gap is a single visual language for both equations and sound.

2. Reference Poles and What They Mean

Lucio Arese / TouchDesigner direction

Lucio Arese is a useful reference for the design philosophy side of the project: audio and birdsong are not just used as triggers for generic effects, but are translated into structured visual models based on acoustic properties. That suggests a stronger product identity than a standard music visualizer.

Useful source: [Lucio Arese YouTube videos](#) and public Instagram reels discussing spectral centroid, brightness, and other structured acoustic features.

Sebastian Lague fluid simulation direction

The YouTube link provided is Sebastian Lague’s fluid simulation video, not Lucio Arese. This matters because it points to a second inspiration category: visually compelling mathematical

simulation. That is a useful reference for later phases of the app, especially if the project expands from waveform visualization into field and system simulations.

Useful source: [Sebastian Lague - Coding Adventure: Simulating Fluids](#)

3. Market Map: Existing Tools and What to Borrow

Desmos

Fast graphing, sliders, animation, immediate equation feedback.

- Borrow the frictionless input model, instant graph updates, and parameter sliders.

Source: <https://www.desmos.com/calculator>

GeoGebra

Graphing plus geometry, 3D, and broader math exploration.

- Borrow the multi-mode math exploration idea, especially if 3D surfaces or parametric systems are added later.

Source: <https://www.geogebra.org/graphing?lang=en>

TouchDesigner / Derivative tutorials

Node-based, real-time, visually rich audio-reactive pipelines.

- Borrow modular signal flow thinking, instancing, and audio-driven geometry.

Source: <https://derivative.ca/community-post/tutorial/audio-reactive-visualizers-touchdesigner/72251>

Magic Music Visuals

A mature modular visualizer with audio/MIDI reactivity and customizable 2D/3D visuals.

- Borrow the “visual instrument” mindset and strong live-performance orientation.

Source: <https://magicmusicvisuals.com/features>

Browser-based Synesthesia projects

Examples of web-based audio visualizers using Three.js and Web Audio/FFT.

- Borrow the browser-first architecture idea if a desktop app is too heavy for version 1.

Source: <https://github.com/tigerwins/Synesthesia/>

Processing Sound library

An approachable environment for synthesis, playback, and signal analysis.

- Borrow the rapid prototyping mindset for exploring sound-analysis-to-visual mappings.

Source: <https://processing.org/reference/libraries/sound/>

Falstad math and physics applets

Interactive phenomenon sandboxes covering waves, oscillation, and physics.

- Borrow the “phenomenon lab” framing for later expansion into educational simulation templates.

Source: <https://www.falstad.com/mathphysics.html>

4. White Space / Opportunity

There does not appear to be a common polished product that does all of the following together: takes equations or waveforms as direct input, plots them conventionally, lets them drive live visuals, and then extends that same logic into simulation templates across math, physics, and audio. That is the clearest opening.

- Desmos and GeoGebra are mathematically clear but are not immersive visual instruments.
- TouchDesigner and Magic Music Visuals are visually expressive but are not designed around equation-first exploration.
- Simulation projects and educational applets are useful references but usually are not built as creative, real-time visualization tools for math input.

5. Recommended Product Direction

Best framing for version 1

A real-time equation and waveform visualizer that turns math into motion.

Product description draft

An interactive math and audio visualization app where users enter equations, waveforms, or parameterized systems and instantly see both the underlying graph and a real-time visual interpretation. The goal is to turn mathematical structure into motion, making phenomena like oscillation, interference, harmonics, resonance, and parametric geometry feel explorable rather than abstract.

Two operating modes

- Mode A - Math Input: enter an equation, parametric curve, or system; graph it; animate parameters; render a visual scene from the same data.
- Mode B - Audio Input: use a generated tone, uploaded file, or live input; analyze waveform and spectrum; drive the same visual engine from signal features.

This gives the app one coherent concept: one visual language for equations and sound.

6. MVP Boundaries

What version 1 should include

- Single-variable function input and plotting.
- Basic waveform presets: sine, square, triangle, sawtooth.
- Parameters for amplitude, frequency, phase, offset, and damping where relevant.
- Real-time graph updates with sliders.
- At least one strong live visualization scene driven by the same math/signal.
- Optional audio generation for supported waveforms.
- A small gallery of curated presets that look good immediately.

What version 1 should not try to do

- Do not try to simulate every branch of physics or mathematics.
- Do not start with fluid simulation, PDE-heavy systems, or a giant library of phenomena.
- Do not begin with a full node-based editor unless it falls out naturally from the build.
- Do not optimize for every classroom use case on day one.

Best first phenomenon families

- Waveform lab: sine, square, triangle, sawtooth, envelope changes.
- Fourier and harmonic composition: stacking sinusoids into composite signals.
- Oscillation and resonance: damped oscillators, beats, interference, standing-wave-like behavior.
- Parametric and polar math art: Lissajous curves, spirals, rose curves, and parameterized geometric motion.

7. UX and System Ideas

- Left panel: equation/waveform input and parameter controls.
- Center panel: conventional graph view.
- Right or full-stage panel: cinematic real-time visualization scene.
- Preset selector for “oscillation,” “harmonics,” “Lissajous,” “resonance,” and “audio reactive.”
- Feature extraction layer for waveform and audio: amplitude envelope, FFT bins, dominant frequency, spectral centroid, zero crossings, phase, and harmonics.
- Mapping layer that converts features into geometry, particle motion, light intensity, rotation, camera movement, or shader parameters.

What makes it feel serious instead of gimmicky

- Map meaningful mathematical or signal features to visuals, not just volume to scale or bass to brightness.
- Preserve the conventional graph view so the user can connect beauty back to the underlying math.
- Keep a small set of polished visualization modes rather than too many weak ones.
- Make presets educational enough to explain and artistic enough to demo.

8. Differentiators to Emphasize

- Equation-first rather than beat-first.
- Math graph plus live visual scene, not one or the other.
- One rendering language shared between direct math input and direct audio input.
- Useful for creative coding, demos, math intuition, and portfolio presentation.
- Potential expansion path into physics simulation, generative art, and educational tools.

9. Claude Prompt Pack

Primary prompt to paste into Claude

You are helping me design and implement a math and audio visualization app. This is a research-and-planning-guided build, not a generic code dump.

Project concept:

I want an app where a user can enter equations or waveforms and instantly get both:

- 1) a standard mathematical graph, and
- 2) a real-time visual scene driven by the same function, waveform, or extracted signal features.

Core product goal:

Build version 1 as a real-time equation and waveform visualizer that turns math into motion.

Do NOT frame this as a giant “simulate all of math/physics” platform.

Product boundaries:

- Prioritize equation-first and waveform-first exploration.
- Focus on functions, waveforms, oscillation, harmonics, interference, resonance, and parametric curves.
- It should feel visually impressive, but still mathematically grounded.
- Preserve a conventional graph view alongside the artistic visualization.
- Keep the architecture extensible for future simulation templates.

Inspiration references:

- Desmos / GeoGebra for instant graphing and sliders
- TouchDesigner / Lucio Arese style structured audio-reactive visuals
- Sebastian Lague style real-time simulation aesthetics as a later inspiration, not the first core feature
- Magic Music Visuals / browser visualizers for modular audio-reactive design ideas

What I want from you:

- 1) Propose the best architecture for an MVP.
- 2) Recommend the best stack for this app, prioritizing strong visuals and reasonable development speed.
- 3) Compare desktop vs browser implementation for this use case.
- 4) Define the MVP features and explicitly separate them from future features.
- 5) Suggest a clean UI layout.
- 6) Explain how equations/waveforms should map into the visual engine.
- 7) When giving code, keep it modular, readable, and easy to extend.
- 8) When uncertain, prefer simple robust systems over fancy fragile ones.

Rules:

- Do not over-scope.
- Do not introduce unnecessary frameworks unless they clearly help.

- Do not remove the graphing side in favor of only visual effects.
- Do not give me filler. Be concrete and opinionated.
- If you propose a feature, explain why it belongs in MVP or why it should wait.
- Whenever possible, give step-by-step build order.
- Flag technical risks early.
- Assume I want a portfolio-worthy result, not just a toy demo.

Current priority:

Start by giving me:

- A) the best product definition,
- B) the best tech stack options,
- C) the recommended MVP feature list,
- D) the recommended file/project structure,
- E) the first implementation milestones.

Optional follow-up prompt once Claude responds

Now refine the plan into a concrete implementation roadmap.

Requirements:

- Pick one stack and justify it.
- Give me a milestone-based build plan.
- For each milestone, list deliverables, core files, and acceptance criteria.
- Keep version 1 limited to equation input, waveform presets, real-time graphing, and one polished visual scene.
- Any advanced simulations or fluid-like systems should be listed as phase-2 or phase-3 ideas unless absolutely necessary.

Output format:

- 1) Chosen stack
- 2) Why this stack wins
- 3) MVP scope
- 4) Deferred scope
- 5) Project folder structure
- 6) Milestone 1 through Milestone N
- 7) Risks and mitigation
- 8) First coding task to start immediately

10. Rules for the Claude Workflow

- Use this document as the project brief and source of scope boundaries.
- Ask Claude to commit to one recommended stack instead of giving five equal options forever.
- Keep asking Claude to separate MVP from phase 2 and phase 3.
- When Claude gives code, request small modular milestones instead of a huge monolith.
- Have Claude explain why each file exists and how it connects to the overall architecture.
- When visuals are discussed, ask for concrete signal-to-visual mappings instead of vague “audio reactive” language.
- Treat Lucio Arese as inspiration for structured audio-to-visual thinking, not as a requirement to copy TouchDesigner exactly.
- Treat Sebastian Lague as inspiration for simulation aesthetics and numerical systems, mainly for later expansion.

11. Strategic Recommendation

This project is stronger as a portfolio piece than a generic LLM or RAG app because it aligns better with quantitative, simulation, and visual-thinking strengths. It also offers a richer story: mathematical modeling, signal analysis, rendering, UI design, and real-time systems all in one project. The most important discipline is scope control. A polished waveform/equation visualizer is far more valuable than a broad but unfinished simulation platform.

12. Sources Appendix

- Lucio Arese YouTube videos: <https://www.youtube.com/@lucioarese/videos>
- Lucio Arese-related public Instagram reel example surfaced in search: <https://www.instagram.com/reel/DXHqNawjSVU/>
- Sebastian Lague - Coding Adventure: Simulating Fluids: <https://www.youtube.com/watch?v=rSKMYc1CQHE>
- Desmos Graphing Calculator: <https://www.desmos.com/calculator>
- GeoGebra Graphing Calculator: <https://www.geogebra.org/graphing?lang=en>
- Derivative audio-reactive tutorial listing: <https://derivative.ca/tags/audio-reactive>
- Derivative tutorial - Audio Reactive Visualizers in TouchDesigner: <https://derivative.ca/community-post/tutorial/audio-reactive-visualizers-touchdesigner/72251>
- Magic Music Visuals features: <https://magicmusicvisuals.com/features>
- Processing Sound library reference: <https://processing.org/reference/libraries/sound/>
- Synesthesia browser-based 3D music visualizer repository: <https://github.com/tigerwins/Synesthesia/>
- Synesthesia web app example: <https://github.com/BiancaDogareci/Synesthesia-Web-App>
- Falstad math and physics applets: <https://www.falstad.com/mathphysics.html>